合肥工业大学计算机与信息学院

计算机组成原理实验教程

即

基于 Verilog HDL 的 CPU 设计实验教程

修改稿

2023.4.8

目录

实验一 Vivado平台入门 3

实验二 多路选择器的实现 6

实验三 CPU 部件实现之 ALU 和寄存器堆 18

实验四 CPU 部件实现之PC 和半导体存储器 RAM 19

实验五 单周期 CPU 设计与实现——单指令 CPU 20

实验六 单周期 CPU 设计与实现——十条指令 CPU 28

实验七 多周期 CPU 设计与实现 30

实验一 Vivado平台入门

1. 实验内容

   完成 Vivado 的安装

2. 实验目标

   1）安装 Xilinx 的 Vivado 开发套件。

   2）熟悉 Vivado 仿真，原理图，下载方法。

3. 实验步骤

   登录http://china.xilinx.com/products/design-tools/vivado.html

   选择合适的版本下载。（下载需要Xilinx账户，请自行申请，后续授权也要用到。）

   安装过程不再赘述。

Vivado平台在工程文件完成编码后一般沿着综合、仿真、链接端口、生成Bit文件和下载至开发板的过程完成工程的实践。如**图1.1**所示，展现了打开一个Vivado工程的示例。

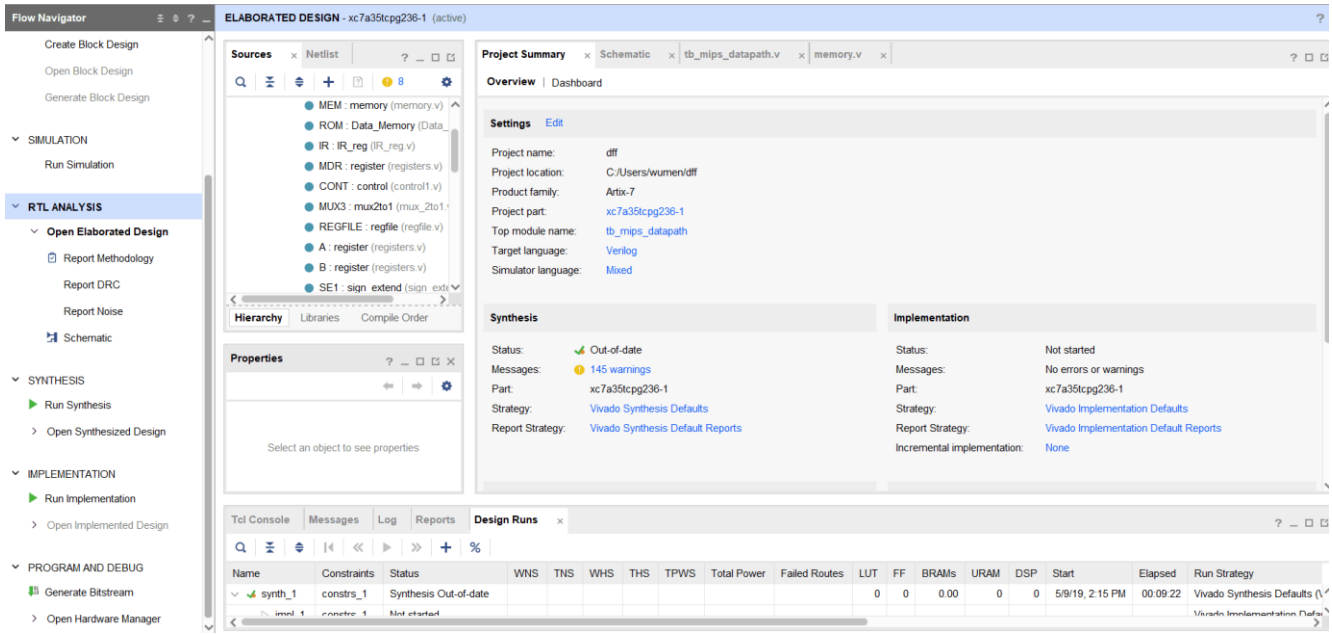


图1.1 Vivado主界面

Flow Navigator：设计流程导航；

Simulation对顶层文件仿真，可以看到时序波形的变化；

RTL Analysis：进行设计规则检测（DRCs），其下的Schematic可以生成器件连线的RTL原理图；

Synthesis是对指定的顶层文件进行层次化综合；

Implementation是对设计进行实现，链接相关端口；

Program and Debug：对bitstream进行设置，Generate Bitstream生成bit流文件，可以打开硬件管理器，将bit流文件烧写到指定设备。如果设计中加入了Debug核，这里会打开Vivado逻辑分析仪，对系统进行Debug。

例如：点击红框位置进行Run Implementation操作。

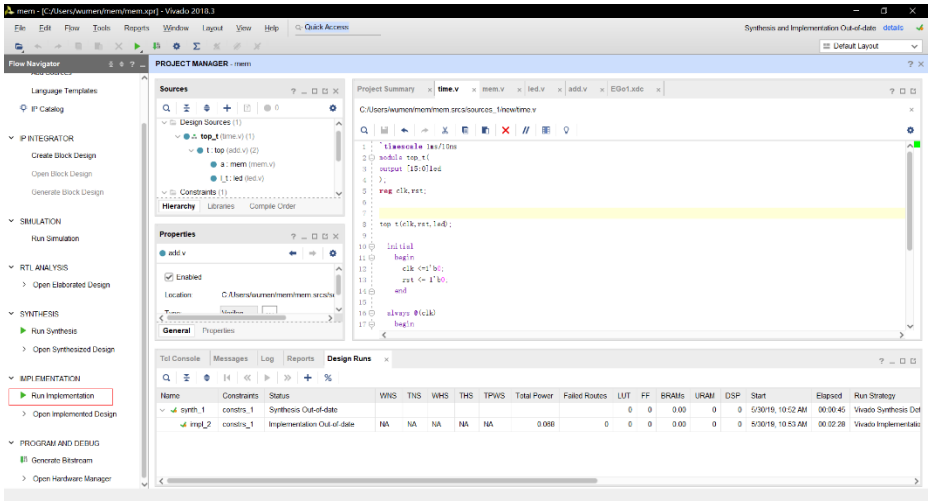


图1.2 Run Implementation操作

在这两部分设计验证都没有问题的情况下，可以生成bit文件，该文件就是用于写入开发板的部分。

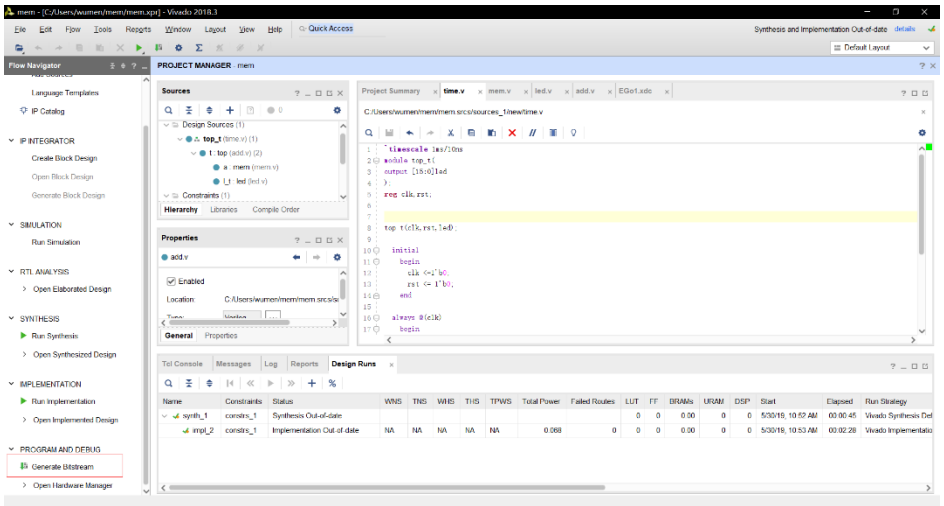


图1.3 Generate Bitstream

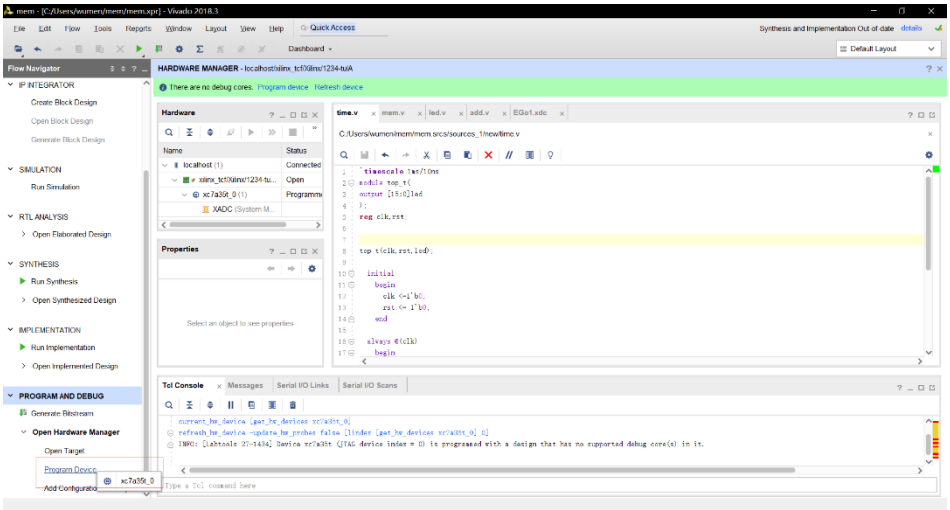


图1.4 Program Device

生成bit文件后就能打开硬件向导，连接开发板后就可以进行下载运行。

实验二 多路选择器的实现

一. 实验内容

这个实验主要使用Vivado软件进行多路数据选择器的设计与实现。多路选择器是现代RISC CPU内部数据通用选择的重要部件，是基于总线的数据通路的一部分。

二. 实验目标

1. 了解Vivado平台开发环境，学习Verilog HDL语言。
2. 掌握多路选择器的工作原理和逻辑功能。
3. 了解计算机条件指令的实现。

三. 实验步骤

1. 建立新工程；
2. 设计代码与输入；
3. 代码综合
4. 软件仿真
5. 硬件约束与实现
6. 生成流代码与下载

四. 实验原理

数据选择器是计算机逻辑电路设计中最重要的基本逻辑电路之一，也是基于数据选择数据通路CPU的重要部件。在Verilog硬件描述语言中可以用if…else或 case语句来生成一个数据选择器。

以2选1和4选1选择器为例，**图2.1**和**图2.2**为多路选择器的原理图：

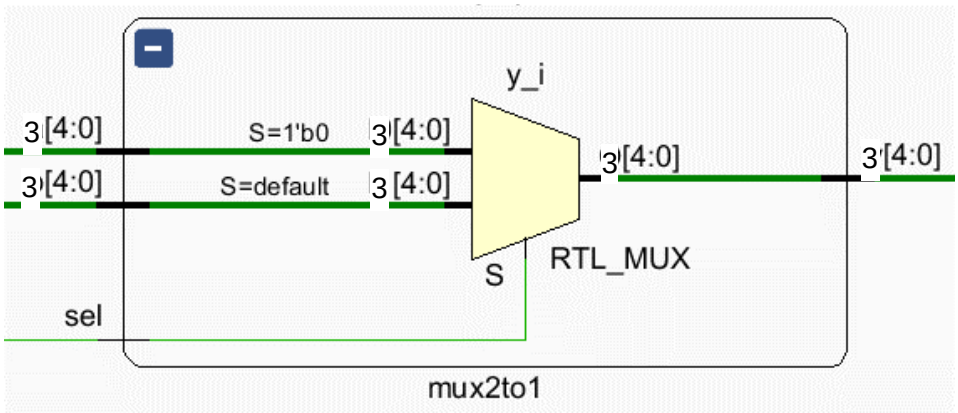


图2.1 2选1

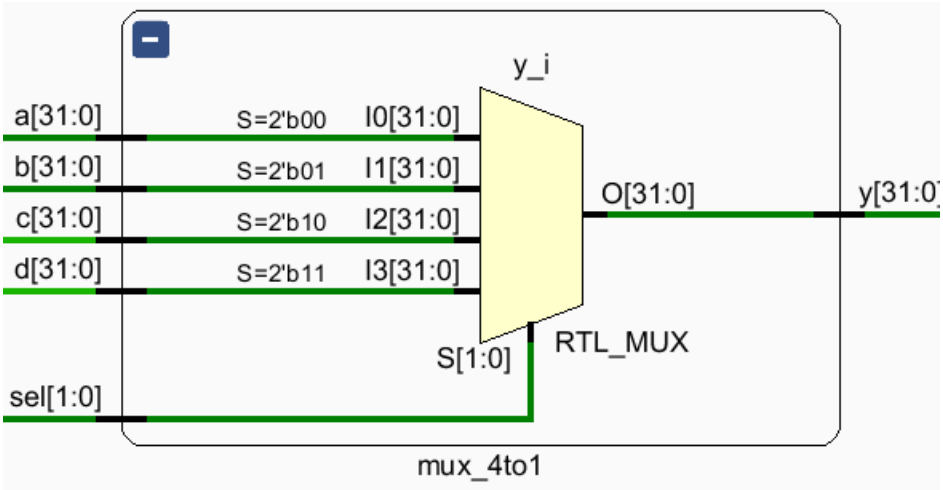


图2.2 4选1

五. 实验步骤

5.1 创建工程

（1）双击桌面Vivado快捷图标 ，启动Vivado。

（2）Create Project进入New Project向导，如图2.3所示。

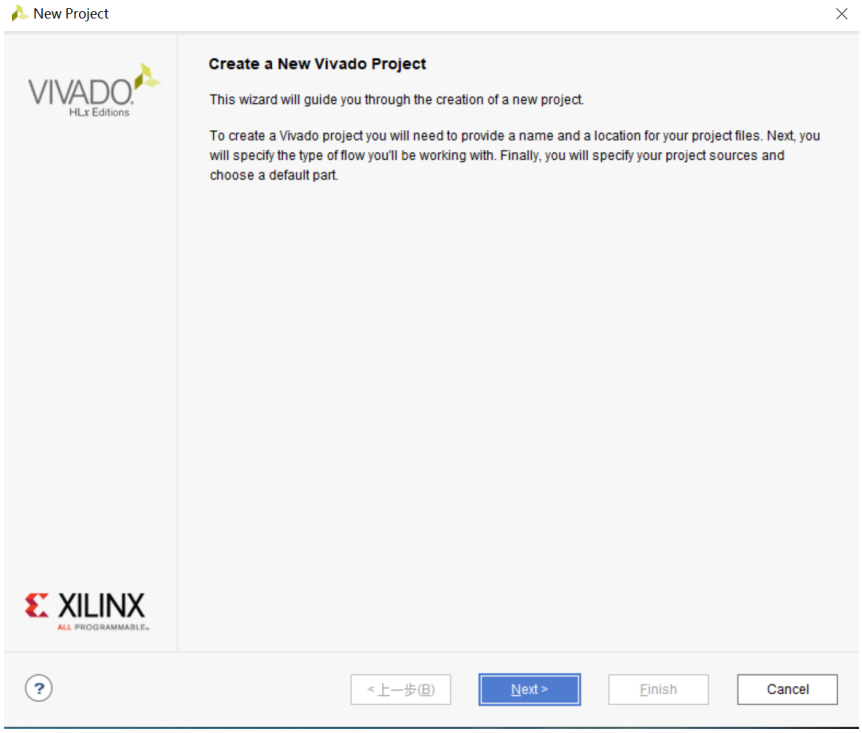
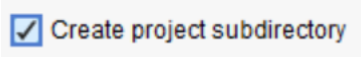


图2.3 New Project

在Project Name中勾选 

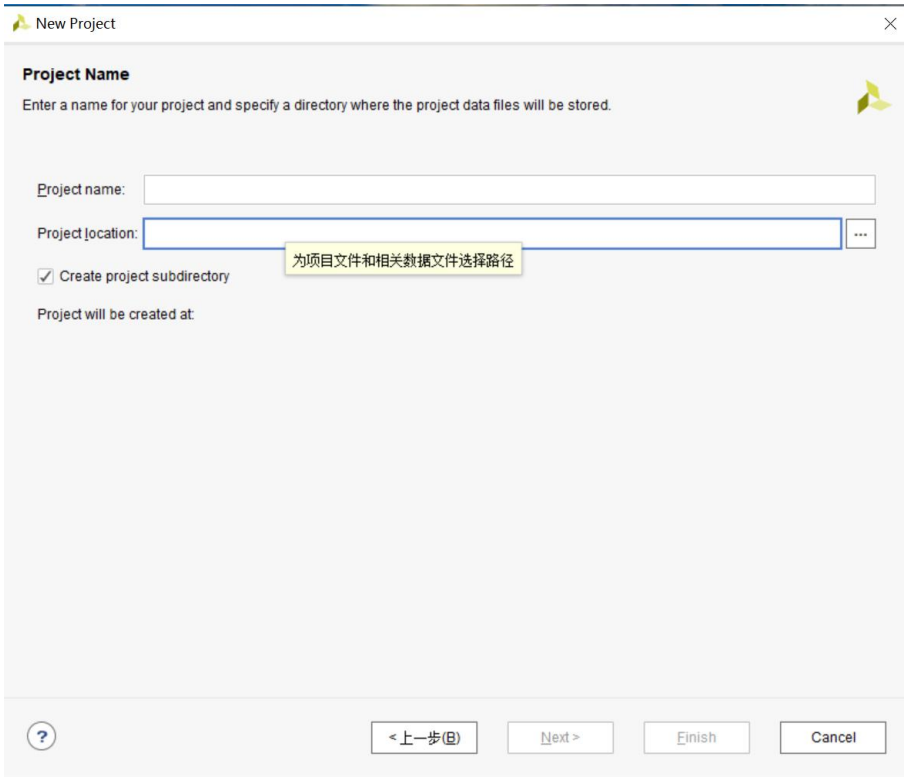


图2.4 Project Name

在Project Type 中选RTL Project，如图2.5

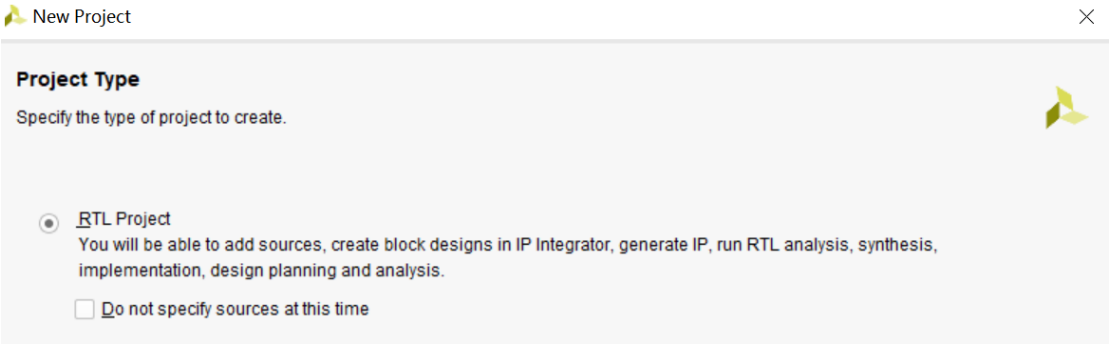


图2.5 Project Type

在Add Sources中选Verilog作为目标语言，也可以选自己熟悉的其他语言，如图2.6。如果有源文件，选Add File，我们没有，所以选Create File，如图2.7。

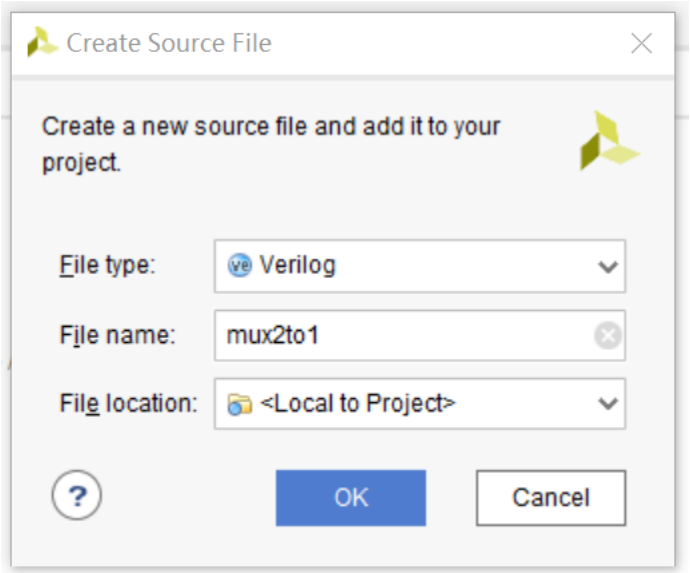


图2.6 Add Sources

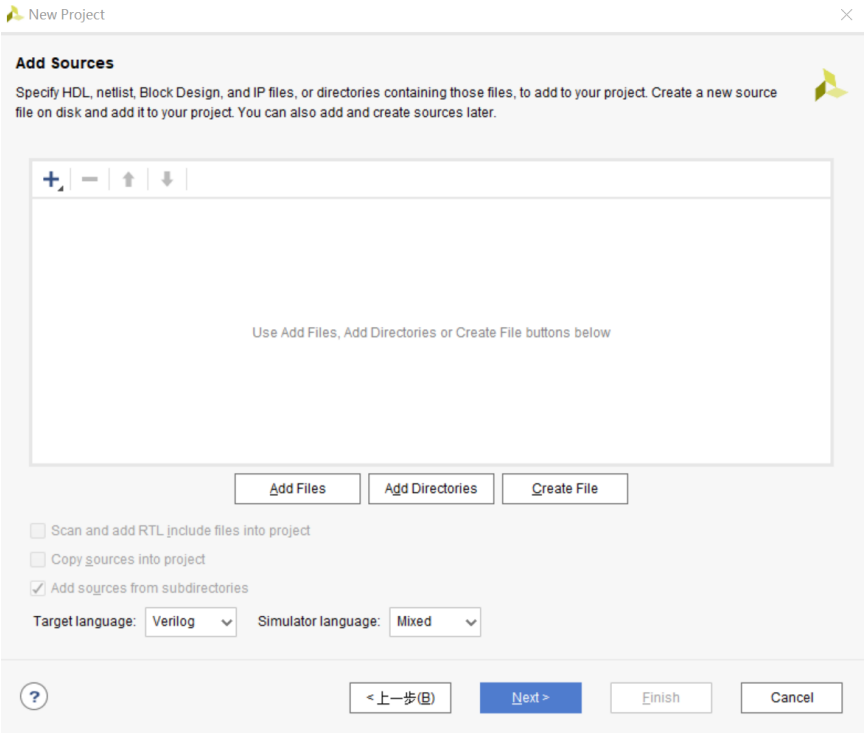


图2.7 Create File

在Add Constraints里单击Next。

在Default Part对话框中选择相应的芯片型号或者硬件平台。我们假设使用EGO1开发平台，如图2.8

2.8 EGO1开发板及芯片规格

具体设置如**图2.9**所示。

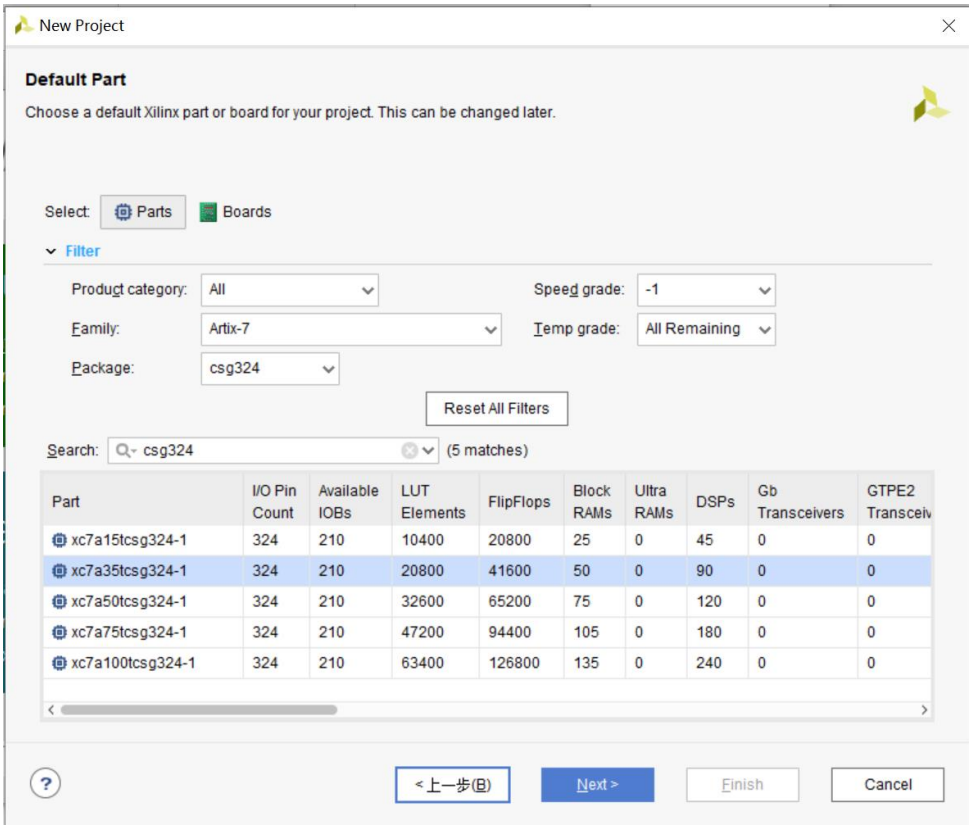


图2.9 Default Part

Define Module中Module Name栏用于输入模块名，下面的列表框用于端口的定义。Port Name表示端口名称，Direction表示端口方向(可选择为input、output或inout)，MSB表示信号最高位，LSB表示信号最低位，对于单信号的MSB和LSB不用填写。当然，端口定义这一步也可以略过，在源程序中再行添加。如**图2.10**.

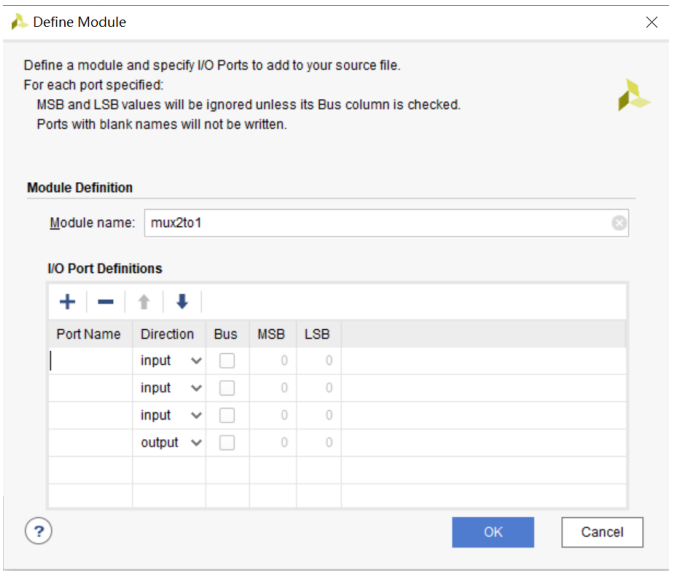


图 2.10 Define Module

定义了模块的端口后，单击Next进入下一步，点击Finish完成创建，进入主界面。

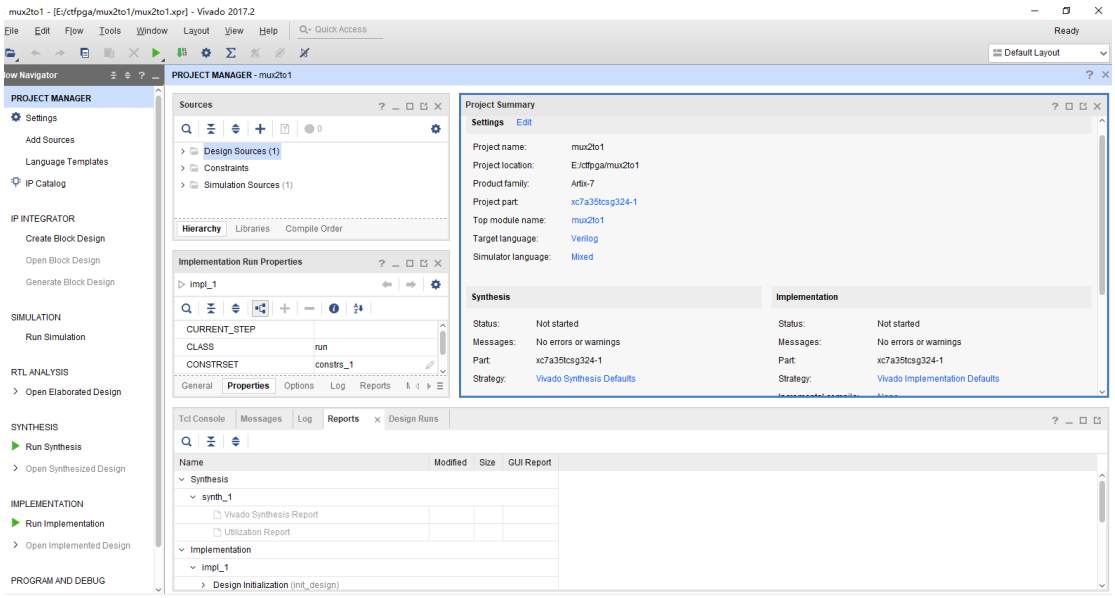


图2.11 Vivado 工程界面

Vivado IDE 主要工作环境包括菜单栏、工具栏、设计流程导航、数据窗口区、工作空间、结果显示区。

(3) 添加或者创建设计文件，如**图2.12**所示

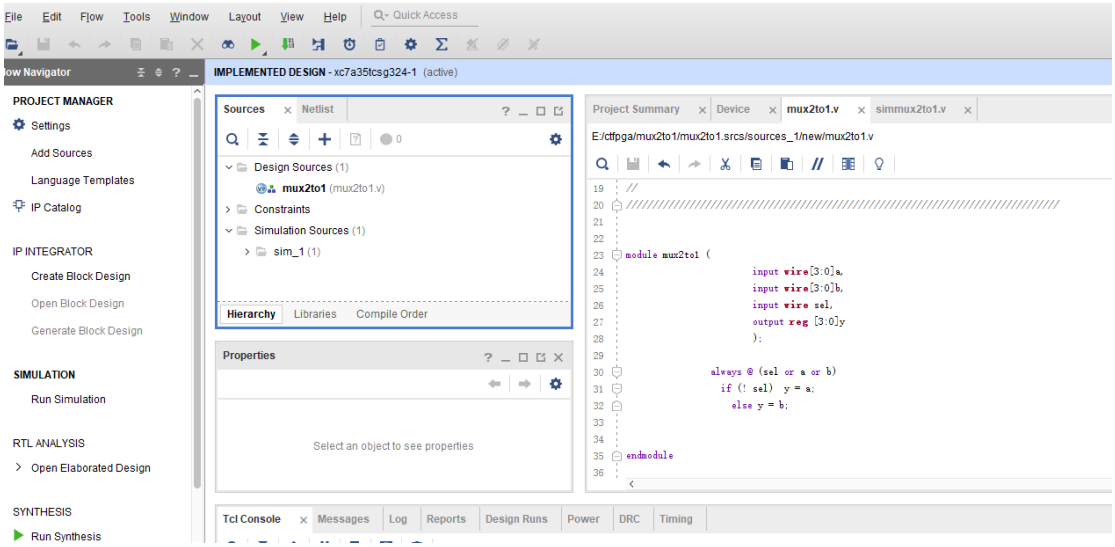


图2.12 工程设计文件输入

2选1的基本代码：

```
module mux2to1 (
                    input wire[3:0]a,
                    input wire[3:0]b,
                    input wire sel,
                    output reg [3:0]y
                    );

                    always @ (sel or a or b)
                    if (! sel)  y = a;
                      else y = b;
endmodule
```

请仿照以上例子自己设计4选1或8选1多路选择器。

5.3 添加约束文件

对于工程中输入的源代码，需要给设计添加管脚和时序约束。管脚约束是将设计文件的输入输出信号设置到器件的某个管脚，（包括设置此管角的电平标准、电流标准、上下拉特性）。时序约束的作用是为了提高设计的工作频率和获得正确的时序分析报告。

在这个实验中，我们将这一步略去。

5.4 综合与实现

综合就是针对输入设计以及约束条件，按照一定的优化算法进行优化处理，获得一个能满足预期功能的电路设计方案。在FPGA设计时，可以用硬件描述语言或者是原理图形式来表示电路的功能。综合工具将这些输入文件翻译成由FPGA内部逻辑资源（逻辑单元，RAM存储单元，时钟单元等）按照某种连接方式组成的逻辑连接（网表），并根据用户的要求生成网表文件，这一过程称为综合过程。方法：Flow Navigator→Synthesis→Run Synthesis进行工程综合。如**图2.13**.

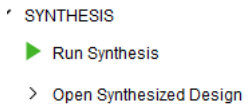


图2.13 综合选项

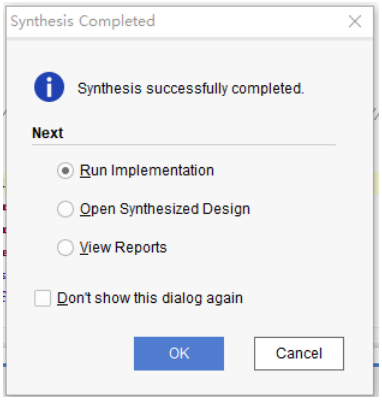


图2.14 综合后的弹出框

实现就是通过翻译、映射、布局布线等过程来完成设计的固化。实现过程首先将综合生成的网表（Netlist）文件,通过翻译变成所选器件的内部资源和硬件单元，如可配置逻辑块（CLB），数字时钟单元（DCM），存储单元（RAM）等，这个步骤称为翻译过程；然后找到对应的硬件关系，将设计与这些硬件资源关系一一对应起来，这又称为映射过程（Map）；最后进行布局布线(Place&Route)，这样设计基本上就可以完全固化到fpga当中了。选择Run Implementation，执行实现过程。

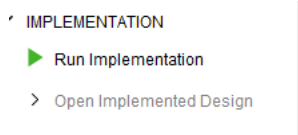


图2.15 实现选项

5.5 仿真并查看波形

（1）添加仿真文件，点图2.16的“+”

图2.16 添加仿真文件

（2）设置仿真文件

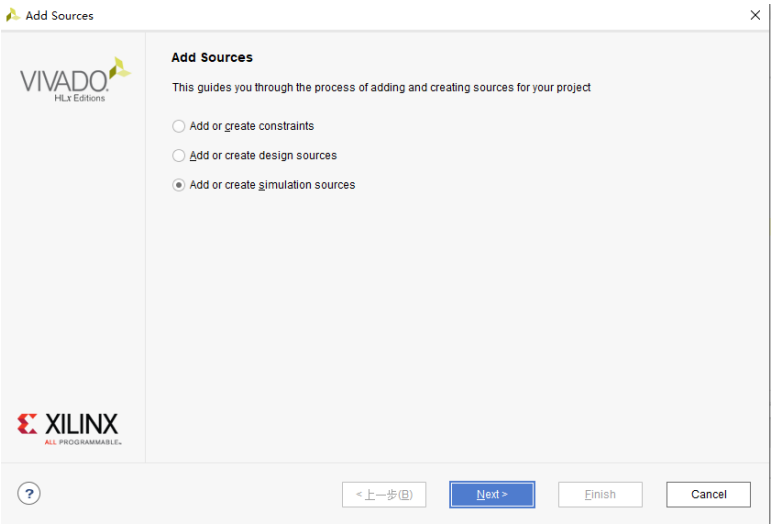


图2.17 Add Source

（3）建立仿真文件

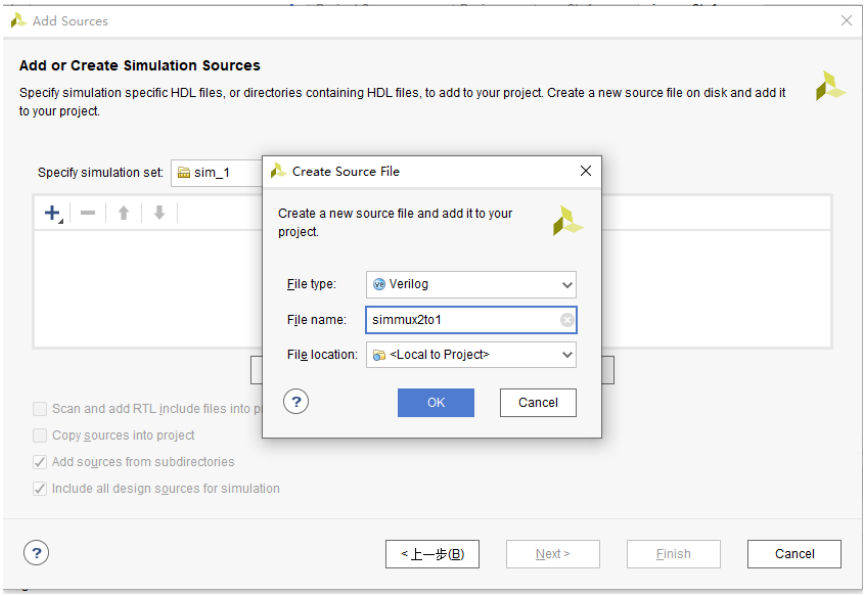


图2.18 建立仿真文件

（4）添加用于仿真的测试程序

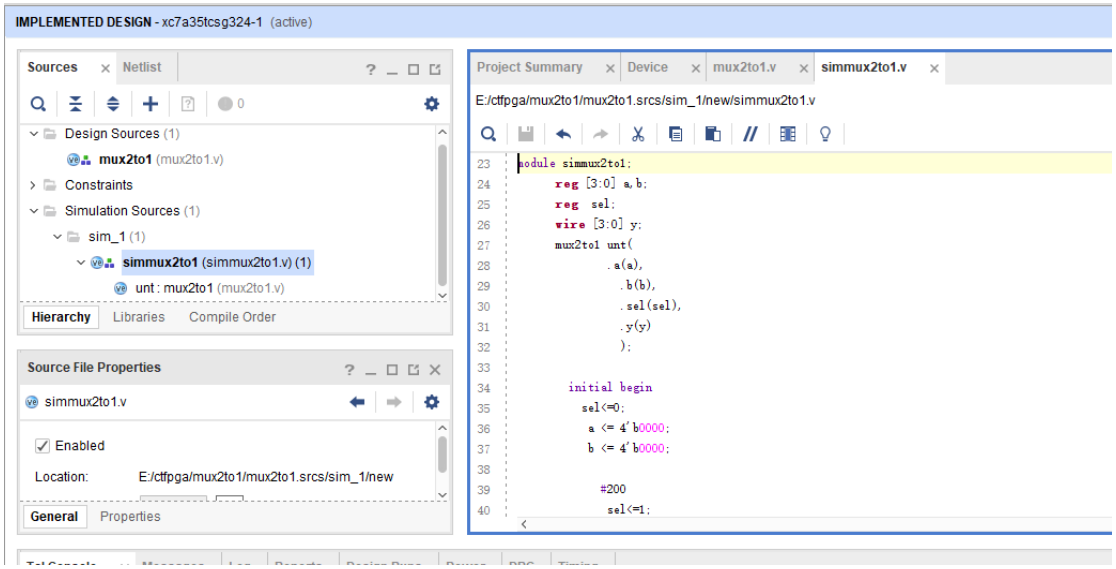


图2.19 编写测试程序

测试代码如下：

```
module simmux2to1;
    reg [3:0] a,b;
    reg  sel;
    wire [3:0] y;
    mux2to1 unt(
          .a(a),
           .b(b),
           .sel(sel),
           .y(y)
           );

          initial    begin
       sel<=0;
        a <= 4'b0000;
        b <= 4'b0000;

         #200
          sel<=1;
          a <= 4'b0001;
          b <= 4'b1000;

           #200
           sel<=0;
           a <= 4'b0010;
           b <= 4'b0100;

            #200
             sel<=1;
             a <= 4'b0011;
             b <= 4'b1100;
```

```
        end

endmodule
```

（5）仿真

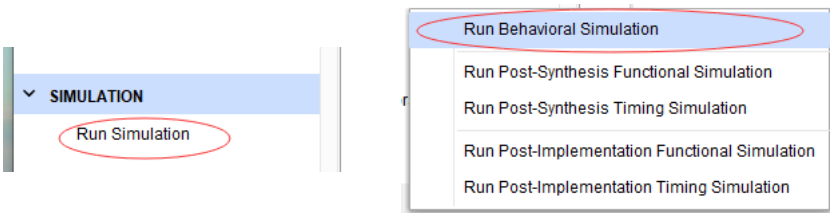
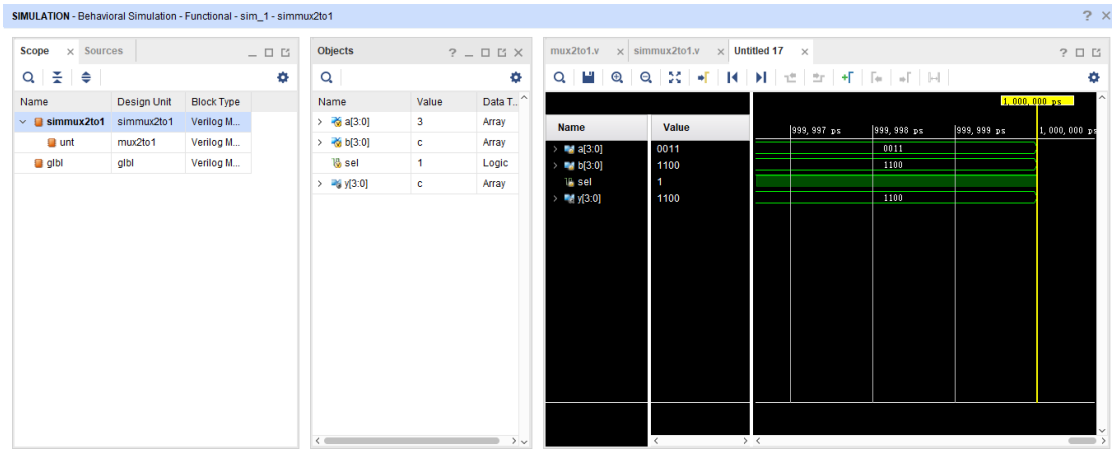


图2.20仿真

（6）分析波形

根据波形中的逻辑值分析是否符合逻辑设计的功能。



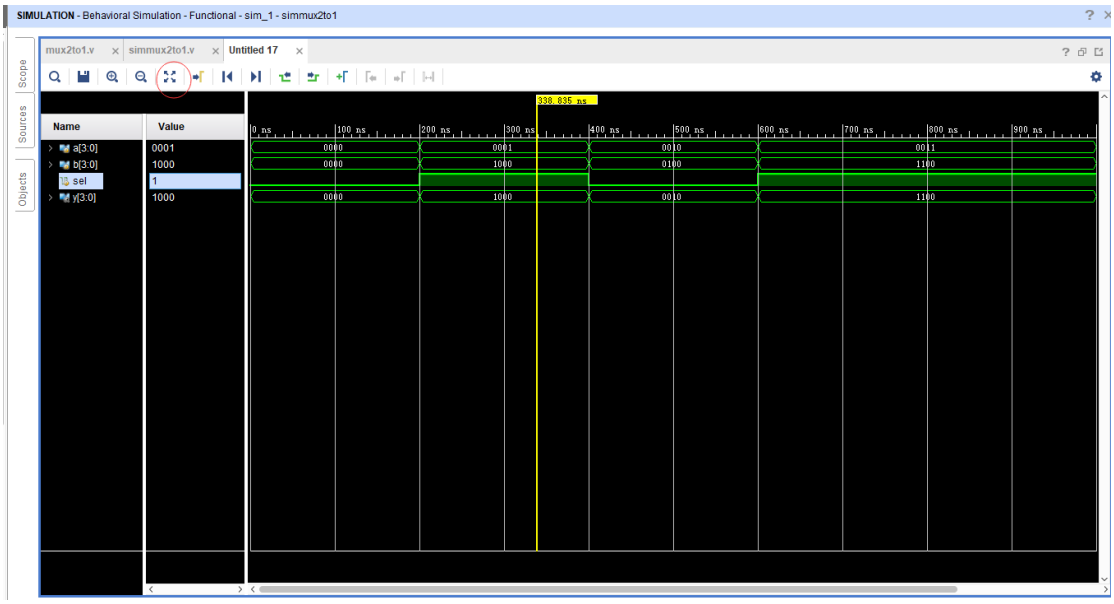


图2.21 分析波形

查看原理图，如图2.22所示。

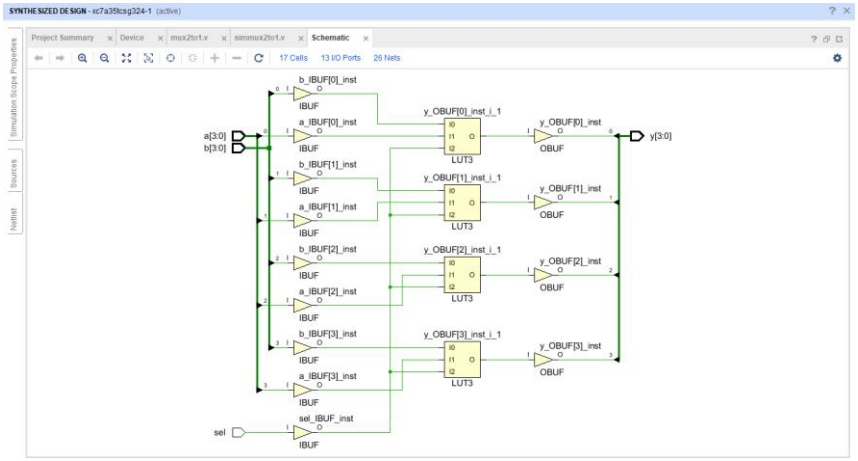


图2.22 原理图

5.6 编译文件及下载

只有编译文件才能配置FPGA，因此在完成综合之后，还要将生成的网表文件转换成可配置的FPGA文件。在实现过程完成后，用户可以查看设计实现的结果或者实现报告。如果需要继续进行生成编译时可选择生成bitstream，点击OK，执行编译过程。（在这个实验中因为没有做约束，也没有开发板用于下载，因此这一步可以跳过。）

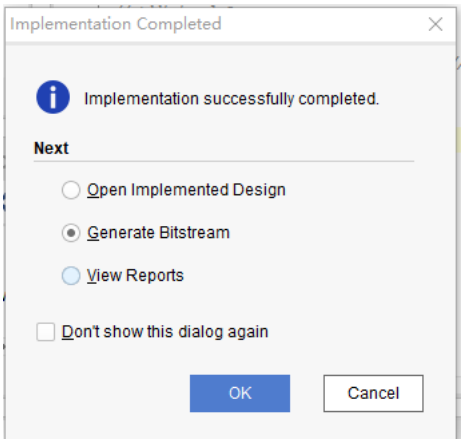


图2.23弹窗

。

实验三 CPU 部件实现之 ALU 和寄存器堆

一、实验目的：

理解和掌握 CPU 中的算术逻辑运算部件（ALU）和寄存器堆（Register File）的工作原理，并使用 Verilog 和 Vivado 进行设计和仿真。

二、实验内容：

1. 使用Verilog 完成 ALU 的设计,并编写测试仿真文件验证其正确性。要求：
   - ALU 支持 16 位的加、减、与、或以及移位运算。
2. 使用Verilog 完成通用寄存器堆的设计，并编写测试仿真文件验证其正确性。要求
   - 寄存器堆包含 8 个 16 位的寄存器；
   - 寄存器堆有两个读端口和一个写端口。

三、实验原理

请根据实验内容要求分别介绍ALU 和寄存器堆的原理图和工作原理

四、实验步骤

参照实验一，依次补充本部分内容。

1. Verilog 关键代码描述
2. 仿真文件描述
3. Vivado 仿真及分析

五、总结

实验四 CPU 部件实现之PC 和半导体存储器 RAM

一、实验目的：

理解和掌握 CPU 中程序计数器 PC 和半导体存储器RAM 的工作原理，并使用Verilog 和 Vivado 进行设计和仿真。

二、实验内容：

1. 使用Verilog 完成程序计数器 PC 的设计，要求：
   - PC 为 8 位计数器
2. 使用Verilog 完成数据存储器的设计，并编写测试仿真文件验证其正确性，要求：
   - 存储字长 16 位，存储容量 1K 字节；
   - 一根读写控制信号线控制读写，低电平有效。

三、实验原理

请根据实验内容要求分别介绍 PC 和RAM 的原理图和工作原理

四、实验步骤

1. Verilog 关键代码描述
2. 仿真文件描述
3. Vivado 仿真（含仿真分析与总结）

五、总结

实验五 单周期 CPU 设计与实现——单指令 CPU

一、实验目的：

通过设计并实现支持一条指令的 CPU，理解和掌握CPU 设计的基本原理和过程。

二、实验内容：

设计和实现一个支持**加法指令**的单周期 CPU。要求该加法指令（表示为 **add r1，r2，r3**）格式约定如下：

- 采用寄存器寻址，r1，r2，r3 为寄存器编号，r1 和 r2 存放两个源操作数，r3 为目标寄存器，其功能为[r1] + [r2] -> r3；
- 指令字长 16 位，操作码和地址码字段分配如下所示：

15 9	8 6	5 3	2 0
OpCode	r1	r2	r3

三、 实验原理

单周期 CPU 是指所有指令均在一个时钟周期内完成的 CPU。CPU 由数据通路及其控制部件两部分构成，因而要完成一个支持若干条指令 CPU 的设计，需要依次完成以下两件事：

1）根据指令功能和格式设计CPU 的数据通路；

2）根据指令功能和数据通路设计控制部件。

3.1 根据功能和格式完成CPU 的数据通路设计

本实验需要设计的CPU 只需要支持一条加法指令，而该指令的功能是在一个时钟周期内从寄存器组中 r1 和 r2 中取出两个操作数，然后送到 ALU 进行加法运算，最后把计算结果保存到 r1 寄存器中。下图给出了该加法指令的数据通路图。

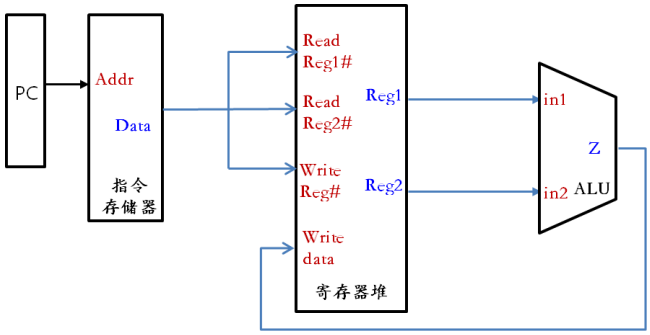


图 5.1 加法指令 add r1,r2,r3 数据通路

此外，还需要确定各个部件的位数，为了简单起见，我们假设目标 CPU 的机器字长、存储字长和指令字长相等均为 16 位，存储单元个数假设为 256，按字寻址，并取 PC 位数为 8。

3.2 根据指令功能、数据通路完成控制单元的设计

控制单元的功能是为当前要执行的指令产生微操作命令从而完成该指令的执行。为了能够完成加法指令的执行，结合**图5.1**，控制单元需要在取出指令后根据指令操作码（本例中是加法指令），控制 ALU（参考实验二）做加法（通过给 alu_op 信号线相应赋值），并把结果写回寄存器组（参考实验三）中（通过给wr_en 赋值为 true）。**图5.2** 给出了整合控制单元后目标CPU 的原理图，系统时钟信号也已标注。

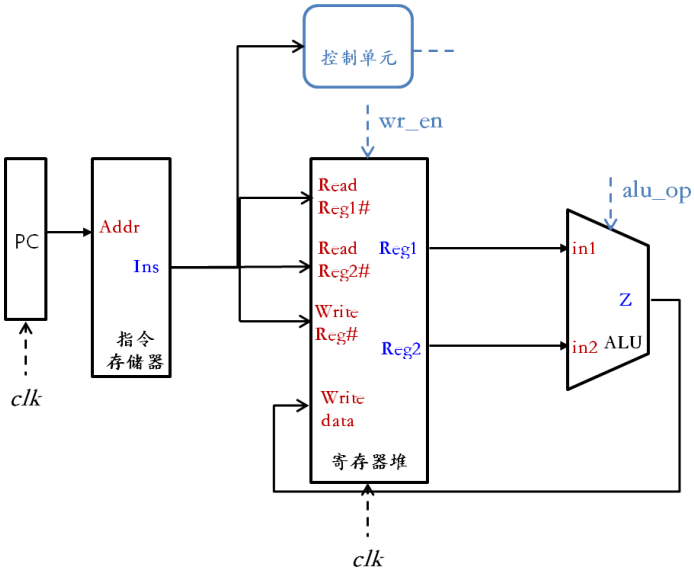


图 5.2 单指令CPU 原理图

四、实验步骤

在第三部分通过对该 CPU 实现细节的分析、设计，并得到该 CPU 的原理图后，就可以依次实现各个模块，并进行仿真验证了。

4.1 CPU 各模块 Verilog 实现

在前面实验中，已经分别设计和实现了 PC、指令存储器、寄存器组和 ALU，这里只给出各个模块的功能描述及其接口定义，具体实现可以直接使用或者调整前面试验的实现代码。

1）PC 模块

表 5.1 PC 模块功能描述

输入	时钟信号 clk、重置信号 rst
输出	指令地址 pc（8 位）
功能	每个时钟上升沿 PC 的值自动加 1，并输出

Verilog 关键代码：

2）指令存储器模块

表 5.2 指令存储器模块功能描述

输入	8 位指令地址Addr
输出	16 位指令 Ins
功能	存放待执行的指令（初始化），并根据地址输出指令

Verilog 关键代码：

3）寄存器堆

表 5.3 寄存器堆模块功能描述

输入	时钟信号 clk、读写控制线wr_en、读寄存器编号 read_reg1 和 read_reg2、写寄存器编号 write_reg、写入数据write_data
输出	对应两个读寄存器编号的寄存器值 reg1 和 reg2
功能	根据读寄存器编号给出对应寄存器的值；在写允许情况下，把写入端的数据在 clk 下降沿写到写寄存器编号对应的寄存器

Verilog 关键代码：

4）ALU

表5.4 ALU 模块功能描述

输入	操作数 in1 和 in2、操作选择信号 alu_op
输出	ALU 运算结果Z
功能	根据操作选择信号计算in1 和 in2 的运算结果Z

Verilog 关键代码：

5）控制单元

表5.5 控制单元模块功能描述

输入	指令（操作码）
输出	寄存器堆的读写控制线wr_en、ALU 的操作选择信号alu_op
功能	根据当前指令功能对 wr_en 和 alu_op 赋值

Verilog 关键代码：

4.2 CPU 顶层文件封装实现

通过根据**图5.2** 将以上定义的模块进行连接、封装就得到了目标 CPU，该 CPU 的输入为系统时钟信号clk 和重置信号 reset。

Verilog 关键代码：

4.3 CPU 模拟仿真

为了仿真验证所实现的 CPU，需要定义仿真文件并在仿真文件中对指令存储器和寄存器堆中的相应寄存器的值进行初始化，并通过仿真波形图查看是否指令得到了正确执行。

1）仿真文件关键代码

2、Vivado 仿真及分析

五、总结

通过实验，请思考你认为完成一个CPU 的设计与实现主要由哪几个步骤完成？主要注意事项有哪些？

六、进一步实验

1）在本节实现的单指令CPU 基础上，添加存数指令 st r1,addr，实现一个可以支持加

法和存数指令的CPU，并使用 Vivado 进行仿真验证。指

令 st r1, addr: **[r1] -> mem[addr]**的格式如下：

15　　9	8　　6	5　　0
st	r1	addr

实验分析提示

首先根据新增加的访存指令功能设计其数据通路，如**图5.3**。为使 CPU 能够支持前面的加法指令和该访存指令，只需要将两个数据通路进行合并（即图 3.1 和图 6.1），并最终得到该CPU 的原理图，如**图5.4** 所示。

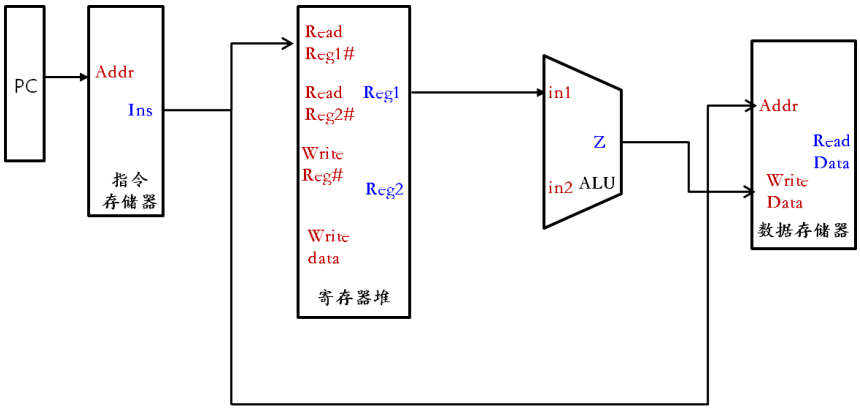


图 5.3 访存指令数据通路

如果要设计支持更多条指令的CPU，只需根据每条指令设计其数据通路，并采用合并的方式构建支持所有指令的 CPU 数据通路，然后在进行控制单元的设计即可。在合并过程中需要注意的是：当某个部件的输入端口有多个输入来源的时候需在此端口前添加一个多路选择器从而允许控制部件根据执行的需要选择所需的数据来源。

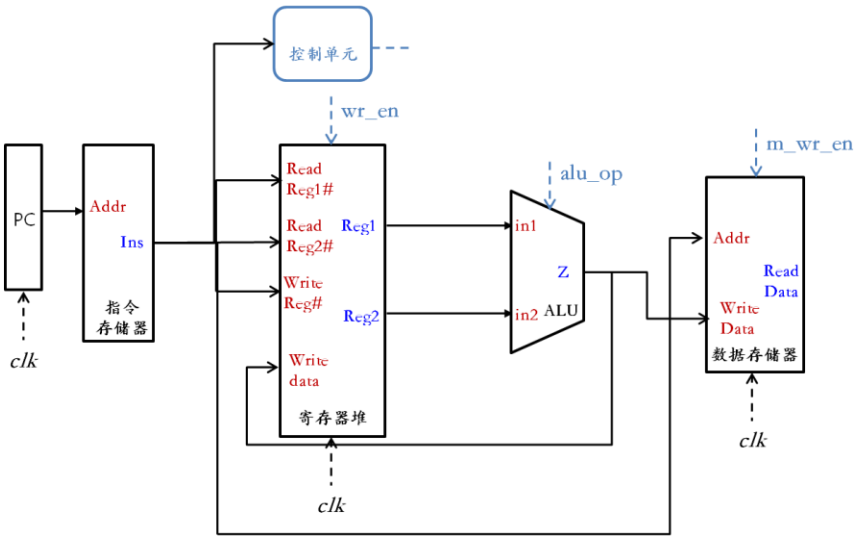


图 5.4 支持加法和访存指令的CPU 原理图

2） 设计和实现一个支持加法 add、立即数加 addi、存数 st、取数 ld 四条指令的 CPU（机器字长=指令字长=存储字长），要求指令格式如下：

- 加法 add: **[r1] + [r2] -> r1**

15 12	11 9	8 6	5 0
add	r1	r2	

- 立即数加 addi r1,Imm: **[r1]+SignExt[Imm] -> r1**

15 12	11 9	8 0
addi	r1	imm

- 存数 st r1, addr: **[r1] -> mem[addr]**

15 12	11 9	8 0
st	r1	addr

- 取数ld r1, addr: **mem[addr] -> r1**

15 12	11 9	8 0
ld	r1	addr

实验六 单周期 CPU 设计与实现——十条指令 CPU

一、实验目的：

通过设计并实现支持 10 条指令的 CPU，进一步理解和掌握 CPU 设计的基本原理和过程。

二、实验内容：

设计和实现一个支持如下十条指令的单周期CPU。

- **非访存指令**

◆ 清除累加器指令 CLA

◆ 累加器取反指令 COM

◆ 算术右移一位指令 SHR：将累加器 ACC 中的数右移一位，结果放回 ACC

◆ 循环左移一位指令 CSL：对累加器中的数据进行操作

◆ 停机指令STP

- **访存指令**

◆ 加法指令 ADD X：[X] + [ACC] –〉ACC，X 为存储器地址，直接寻址

◆ 存数指令 STA X，采用直接寻址方式

◆ 取数指令 LDA X，采用直接寻址

- **转移类指令**

◆ 无条件转移指令 JMP imm：signExt(imm) -> PC

◆ 有条件转移（负则转）指令 BAN X: ACC 最高位为 1 则（PC）+ X -> PC,否则 PC 不变

三、实验原理

参照上次实验，依次给出指令格式定义、数据通路和控制单元的设计，并给出目标 CPU

的原理图。

四、实验步骤

参照上次实验，对实验过程进行描述和分析。

五、总结

实验七 多周期 CPU 设计与实现

一、实验目的：

通过设计并实现支持若干条指令的多周期 CPU，理解和掌握多周期 CPU 设计的基本原理和过程。

二、实验内容

设计一个支持以下三条指令的包含取指、译码（取操作数）和执行三个工作周期的多周期 CPU。

- 加法 add r1,r2: **[r1]+ [r2] -> r1**

15 12	11 9	8 6	0
addi	r1	r2	XX

- 存数 st r1, addr: **[r1] -> mem[addr]**

15 12	11 9	8 0
st	r1	addr

- 无条件跳转 jmp imm: **SignExt[imm] -> PC**

15 12	11 0
addi	imm

三、实验原理

多周期 CPU 是将单条指令的执行划分为若干个工作周期，每个工作周期用一个时钟周期完成，每条指令的具体功能决定了完成该指令需要的工作周期数目。指令执行过程中在工作周期的转换由有限状态机进行实现，控制单元根据指令和其所处工作周期发出为操作命令

信号来控制指令的执行。

类似于单周期CPU 的设计，多周期CPU 的设计也需要完成 1）数据通路设计和 2）控制单元设计。

3.1 数据通路设计

首先对三条指令的功能依次进行分析：

- 加法 add
  - 取指：PC->Imem->IR；PC+1 -> PC；
  - 译码：RegFile[IR.r1]->A；RegFile[IR.r2]->B；
  - 执行：A、B->ALU-> RegFile[IR.r1]
- 存数 st
  - 取指：PC->Imem->IR；PC+1 -> PC；
  - 译码：无；
  - 执行：RegFile[IR.r1]->mem[IR.addr]
- 无条件跳转 jmp
  - 取指：PC->Imem->IR；PC+1 -> PC；
  - 译码：SignExt[IR.imm]->PC

根据三条指令执行分析，可以依次获得各条指令的数据通路，合并后可以得到如**图7.1**的支持该三条指令的多周期 CPU 数据通路图。特别的，相对于单周期 CPU，多周期 CPU 中添加了由红色标注的 IR 和操作数暂存部件A、B。

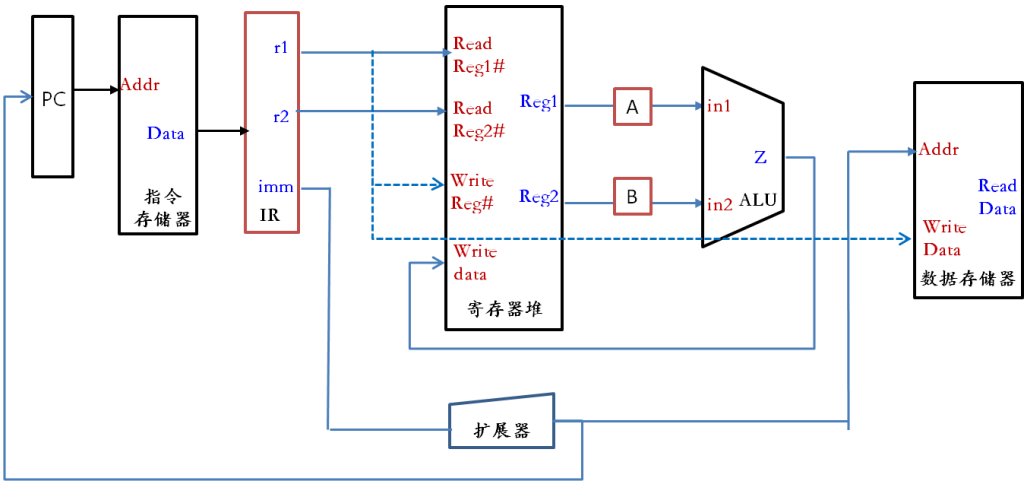


图 7.1 支持三条指令的多周期CPU 数据通路图

3.2 控制单元设计

对于多周期 CPU，由于同一条指令在不同工作周期完成的操作不同，控制单元除了需要根据所执行的指令外，还需要根据其所出工作周期发出控制信号，从而确保指令的正确执行。根据指令功能分析，可以明确控制单元的输入和输出控制信号如下表。

表 7.1. 控制单元的输入输出及说明

输入	Instruction	当前预执行指令
	Period	指令所处的工作周期
输出	PC_Plus_en	使 PC 加 1
	PC_Up_Imm_en	使 PC 根据立即数更新其值
	IR_wr_en	允许 IR 写入
	RegFile_wr_en	允许写入到 Reg. File 中的相应寄存器
	AluOpSel	ALU 执行操作选择信号（或算术逻辑操作选择信号）
	A_B_wr_en	暂存器 A、B 允许写入信号

通过对指令功能的分析，我们可以看到不同指令需要的工作周期数是不一样的，通常采用**有限状态机（FSM）**来对指令所处的工作周期进行实现。**图7.2** 给出了实现本实验中三条指令在各个工作周期的转换的FSM，其中FI、ID、Exe 分别表示取指、译码和执行周期。

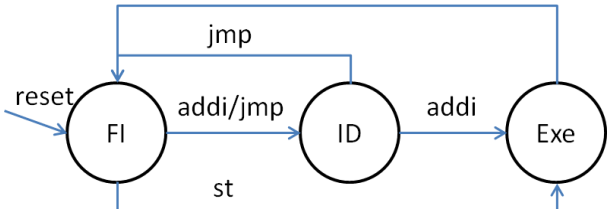


图 7.2. 实现指令执行工作周期（Period）状态转换的 FSM

状态机由状态寄存器（记录当前所处状态的寄存器）、下一状态逻辑（根据当前状态和输入决定下一状态的组合逻辑电路）和输出逻辑（由当前状态和输入决定当前输出的组合逻辑电路）构成，能够根据控制信号按照预先设定的状态进行状态转移，常用于构建转移状态有限的系统。状态机通常分为两类：若输出只和状态有关而与输入无关，则称为 Moore 状态机；若输出不仅和状态有关而且和输入有关系，则称为 Mealy 状态机。**图 7.3** 给出了 Mealy 状态机的原理图，当把输入到产生输出的组合逻辑之间的连线去掉就是 Moore 状态机的原理图了。

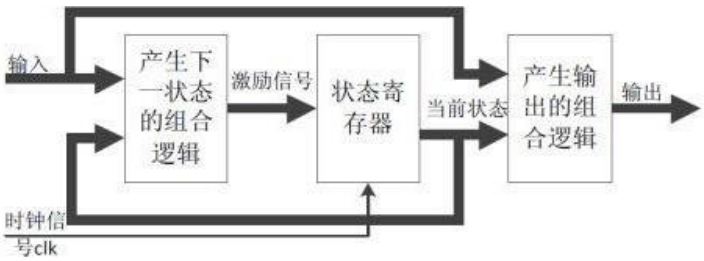


图 7.3 Mealy 状态机原理图

使用Verilog 来描述状态机的方法与描述常规时许电路类似，其设计可以采用三段式设计，也就是根据原理图把状态机的设计分为以下三段：

- 当前状态：采用同步时序方式描述状态转移（如每一个时钟周期触发）；

- 下一状态逻辑：采用组合逻辑方式判断状态转移条件并产生下一状态；
- 输出逻辑：采用组合逻辑方式描述输出。

四、实验步骤

参照上次实验，对实验过程进行描述和分析。

五、总结

六、进一步实验

根据实验五设计的单周期CPU，将其设计为包含取指、译码和执行三个工作周期的多周期CPU。